Deutsche Bank IIT Guwahati

Problem Statements

This document contains the three programming problems, including their descriptions, examples, explanations, constraints, and the required function signature where provided.

Problem 1: Minimum Number of Cars to Remove for a Bridge Crossing

Problem Statement

You are given an integer array **weights** of size **N**, where **weights[i]** represents the weight of the i-th car, and an integer **maxWeight**, representing the maximum total weight the bridge can support.

At most two cars can be on the bridge at any given time. Whenever two cars are on the bridge together, the sum of their weights must not exceed **maxWeight**. Determine the minimum number of cars that must be removed so that all remaining cars can cross the bridge safely, while preserving their original order.

Examples

Example 1

```
Input: weights = [1, 2, 5, 2, 1], maxWeight = 6
Output: 1
```

Explanation: Remove the car with weight 5. The remaining cars can cross without violating the bridge's weight limit.

Example 2

```
Input: weights = [3, 7, 2, 4, 5], maxWeight = 8
Output: 2
```

Explanation: Remove the cars with weights 7 and 5. The remaining cars can cross safely.

Example 3

```
Input: weights = [4, 4, 4, 4], maxWeight = 7
Output: 4
```

Explanation: Any two cars together weigh 8, which exceeds the bridge's capacity. Therefore, every car must be removed if the requirement is that two cars must always be allowed on the bridge simultaneously.

Constraints

- $1 \leq N \leq 10^5$
- $1 \leq \text{weights}[i] \leq 10^9$
- $1 \leq \text{maxWeight} \leq 10^9$

Problem 2: Count Anagrams Without Identical Adjacent Characters

Problem Statement

You are given a string **s** consisting of lowercase English letters (a–z). Count the number of distinct anagrams of **s** in which no two consecutive characters are the same. Two anagrams are considered distinct if their resulting strings are different.

Examples

Example 1

```
Input: s = "aab"
Output: 1
```

Explanation: The distinct anagrams are "aab", "aba", and "baa". Only "aba" satisfies the condition.

Example 2

```
Input: s = "aabc"
Output: 6
```

Explanation: There are 12 distinct anagrams of "aabc". Exactly 6 have no two identical consecutive characters.

Example 3

```
Input: s = "aaa"
Output: 0
```

Explanation: Every permutation contains consecutive identical characters.

Example 4

```
Input: s = "abc"
Output: 6
```

Explanation: All 6 permutations satisfy the condition because every character is distinct.

Constraints

• $1 \leq |s| \leq 50$
• s contains only lowercase English letters (a–z).
• The answer is guaranteed to be at most 2000.

Problem 3: Maximum Sum of Unique Elements Under Index Constraints

Problem Statement

You are given an integer array **arr** of size **N**. Construct a new array **ans** of the same size that satisfies all of the following conditions:

1. **Index-wise constraint:** For every index i, ans[i] ≤ arr[i].
2. **Uniqueness:** All elements in ans must be distinct.
3. **Maximum sum:** The sum of all elements in ans should be as large as possible.

Return the resulting array ans.

Examples

Example 1

```
Input: arr = [3, 3, 3]
Output: ans = [3, 2, 1]
Maximum Sum = 6
```

Explanation: All elements must be unique, and each element must be less than or equal to its corresponding element in arr. Assigning [3, 2, 1] satisfies both conditions and gives the maximum possible sum.

Example 2

```
Input: arr = [2, 2, 1]
Output: ans = [2, 1, 0]
Maximum Sum = 3
```

Explanation: The first element can be 2, but the second cannot also be 2, so it is assigned 1. The third element is assigned 0 to maintain uniqueness.

Example 3

```
Input: arr = [5, 4, 5, 3]
Output: ans = [5, 4, 3, 2]
Maximum Sum = 14
```

Explanation: The resulting array contains only unique elements, and every element is less than or equal to its corresponding element in the original array.

Constraints

• $1 \leq N \leq 10^5$
• $1 \leq arr[i] \leq 10^9$

Function Signature (C++)

```
vector<int> maximumUniqueArray(vector<int>& arr);
```

Objective

Construct an array satisfying all the stated constraints while maximizing the sum of its elements.